

IMPROVING BLOCKCHAIN SCALABILITY: THROUGHPUT ENHANCEMENT IN ZK-ROLLUPS

**Fernando T.H.L. : 210167E
Gamage M.S. : 210176F
Lishan S.D. : 210339J
Perera M.V.D.P.D.S. : 210466U**

**Bachelor of Science Engineering Honours in Computer Science and
Engineering**

**Department of Computer Science and Engineering
University of Moratuwa, Sri Lanka**

June 2026

IMPROVING BLOCKCHAIN SCALABILITY: THROUGHPUT ENHANCEMENT IN ZK-ROLLUPS

**Fernando T.H.L. : 210167E
Gamage M.S. : 210176F
Lishan S.D. : 210339J
Perera M.V.D.P.D.S. : 210466U**

**This final year project report is submitted in partial fulfillment of the
requirements for the Bachelor of Science Engineering Honours in
Computer Science and Engineering**

**Department of Computer Science and Engineering
University of Moratuwa, Sri Lanka**

June 2026

DECLARATION OF THE CANDIDATES AND SUPERVISOR

We declare that this is our own work and this final year project report does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to University of Moratuwa the non-exclusive right to reproduce and distribute our final year project report, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

Fernando T.H.L.:	Date:
Gamage M.S.:	Date:
Lishan S.D.:	Date:
Perera M.V.D.P.D.S.:	Date:

The above candidates have carried out research for the Bachelor of Science Engineering Honours in Computer Science and Engineering final year project report under my supervision.

Name of the supervisor: Dr. Sunimal Rathnayake	
Signature of the supervisor:	Date:

Abstract

Ethereum Layer-1 provides strong security and decentralization but remains constrained by limited throughput. ZK-Rollups offer a Layer-2 scaling solution by executing transactions off-chain and submitting cryptographic validity proofs to Layer-1. However, practical performance of a research prototype depends on complex interactions between batching, sequencing policies, proof generation overhead, and data availability strategies. This report presents a modular ZK-Rollup benchmarking prototype designed to isolate and evaluate these performance factors under controlled synthetic workloads. We explore the design and implementation of the sequencer, executor, submitter, and smart contracts. Through rigorous benchmarking, we analyze the throughput–latency–cost trade-offs across different batching and data availability configurations. The results show that larger batches improve gas amortization but increase finality latency, while the TimeBoost scheduling policy improves fairness under burst-load conditions. EIP-4844 blob-based data availability reduces data publication cost by approximately 70% compared with calldata, though RISC0 proving exhibits step-function latency at segment boundaries.

Keywords: ZK-Rollups, Blockchain Scalability, Data Availability, Zero-Knowledge Proofs, Performance Benchmarking.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our supervisor, Dr. Sunimal Rathnayake, for his guidance, feedback, and continuous support throughout this final year project. We also extend our appreciation to the Department of Computer Science and Engineering, University of Moratuwa, for providing the academic environment and resources required to complete this research.

We are grateful to the academic staff, colleagues, and fellow students who provided helpful suggestions and encouragement during the design, implementation, benchmarking, and report preparation stages of this project.

CONTENTS

Declaration of the Candidates and Supervisor	i
Abstract	ii
Acknowledgements	iii
Table of Contents	iv
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
List of Appendices	xi
1 Introduction	1
1.1 Blockchain Fundamentals	1
1.1.1 Blockchain Structure	1
1.1.2 Consensus Protocols	1
1.1.3 Blockchain Types	2
1.1.4 Scalability Metrics	2
1.1.5 The Blockchain Trilemma	2
1.1.6 The Blockchain Quadrilemma	3
1.1.7 Ethereum Scalability Limitations	3
1.2 Layer-1 vs. Layer-2 Scaling	3
1.3 Rollups and ZK-Rollups	4
1.3.1 Zero-Knowledge Proofs	5
1.3.2 zk-SNARKs, zk-STARKs, and RISC Zero	5
1.4 Data Availability (DA)	6

1.5	Motivation	6
1.6	Problem Statement	7
1.7	Research Questions	8
1.8	Research Objectives	9
1.8.1	Mapping Research Objectives to Evaluation Metrics	9
1.9	Project Scope and Assumptions	10
1.9.1	Research Question Mapping	10
1.10	Contributions	11
1.11	Document Structure	11
2	Literature Review	12
2.1	Broad Scalability Taxonomy	12
2.1.1	Write-Performance Scalability	12
2.1.2	Read-Performance Scalability	12
2.1.3	Storage Scalability	13
2.2	ZK-Rollup Bottleneck Research	13
2.2.1	Sequencer Policies and Batching	13
2.2.2	Prover Overhead	13
2.2.3	Data Availability (DA) Dynamics	13
2.2.4	Summary of Existing Literature	14
2.3	Rollup Benchmarking and Public Datasets	14
2.4	Synthesis and Positioning of This Work	15
3	Methodology / System Design	17
3.1	High-Level Architecture Overview	17
3.1.1	Prototype vs. Production Rollup Comparison	17
3.2	Transaction Lifecycle and Data Flow	18
3.3	Component Interactions	19
3.4	Data Flow and Control Flow	20
3.5	Design Rationale	20
3.6	Prototype vs. Production Distinctions	21
3.7	Research Design	21
3.8	Workload Model	22
3.8.1	Why a Local Hardhat Node and Mock Prover Are Used	22
3.9	Configuration Spaces	23
3.9.1	Batching Policy Space	23
3.9.2	Sequencing Policy Space	23

3.9.3	Data Availability Modes	23
3.10	Analytical Cost Model	23
3.11	Evaluation Metrics	24
3.12	Experimental Procedure and Hypotheses	25
4	Implementation	26
4.1	Implementation Overview	26
4.2	Workload Generator	26
4.3	Sequencer	27
4.4	Executor	28
4.5	Prover and Benchmark-Mode Delay	29
4.6	Submitter	29
4.7	Smart Contracts	30
4.8	State Synchronization and Nonce Handling	31
4.9	Benchmarking and Data Collection Tools	31
4.10	Data Tools	31
4.11	Configuration Management	32
4.12	Logging and Metrics	32
4.13	Integration Challenges	32
4.14	Development Challenges	32
5	Evaluation and Results	33
5.1	Environment and Orchestration	33
5.1.1	Local L1 Configuration	33
5.2	Software Stack	34
5.2.1	L1 Network Configuration	34
5.3	Configurations and Experimental Variables	34
5.4	Benchmark Scenarios	35
5.5	Metrics Collection	36
5.6	Reproducibility Setup	37
5.6.1	Overview of Evaluation Stages	37
5.6.2	Unsupported and Inconclusive Findings	38
5.7	Effect of Batch Size on Latency and Cost (Stage 1)	38
5.8	Cost Amortization	39
5.9	Queueing Latency (Soft Finality)	39
5.10	Throughput Results and Configuration Anomalies	40
5.10.1	Adaptive Batching Failures (Stage 2)	40

5.10.2	Sequencer Scheduling and Nonce Ordering (Stage 3)	40
5.11	Data Availability Mode Efficiency (Stage 4)	41
5.12	Prover Backend and Real Proof Behavior (Stage 5)	41
5.12.1	Finalization Time (Hard Finality)	42
5.12.2	Queue Backpressure	42
5.13	Summary of Validated Findings	42
5.14	Answering the Research Questions	43
5.14.1	RQ1: Impact of Batch Size on Performance and Cost	43
5.14.2	RQ2: Impact of Sequencer Scheduling Policies	43
5.14.3	RQ3: Cost Implications of Data Availability Mechanisms	44
5.14.4	RQ4: Practical Bottlenecks in Implementation	44
5.15	The Throughput–Latency–Cost Trade-off	44
5.16	Batching Dynamics and Outliers	45
5.16.1	RQ3: Data Availability Implications	45
5.16.2	RQ1: Impact of Batch Size on Cost and Latency	45
5.17	Design Implications and Trade-offs	46
5.17.1	RQ4: Practical Bottlenecks and Trade-offs	46
6	Conclusions and Recommendations	48
6.1	Conclusion	48
6.2	Future Work	49
	Reference List	50
A	Repository Structure	52
B	Build and Run Guide	53
C	Benchmark Execution Guide	55
D	Configuration Reference	56
E	Metrics Output Schema	57
F	Important Algorithms	58

LIST OF FIGURES

D.1	Example Configuration Dump for Sequencer	56
E.1	Example JSON Dump for Submitter Metrics	57

LIST OF TABLES

1.1	Ethereum vs High-throughput L1s	3
1.2	Research Question Mapping	10
2.1	Comparison of Key Literature on Blockchain Scalability and ZK-Rollups	14
2.2	Synthesis of Literature and Project Positioning	15
3.1	Component Responsibility Summary	20
3.2	Prototype vs. Production Rollup Architecture	21
3.3	ZK-Rollup Cost Model [9, 11]	23
3.4	Key Benchmarking Metrics	24
4.1	Implementation Status Table	26
5.1	Experiment Matrix: Stages 1 through 5	35
5.2	Benchmark Scenario Matrix	36
5.3	Metric Definitions	36
5.4	Reproducibility Commands Table	37
5.5	Prototype Scope Boundary	47
5.6	Why Research-Grade Instead of Production-Grade	47
D.1	Key Configuration Parameters	56

LIST OF ABBREVIATIONS

DA	Data Availability
EIP	Ethereum Improvement Proposal
FCFS	First-Come-First-Served
L1	Layer 1
L2	Layer 2
RISC0	RISC Zero
STF	State Transition Function
TPS	Transactions Per Second
ZK	Zero-Knowledge
ZKP	Zero-Knowledge Proof
zkVM	Zero-Knowledge Virtual Machine

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	Repository Structure	52
Appendix B	Build and Run Guide	53
Appendix C	Benchmark Execution Guide	55
Appendix D	Configuration Reference	56
Appendix E	Metrics Output Schema	57
Appendix F	Important Algorithms	58

CHAPTER 1

INTRODUCTION

This chapter establishes the foundational concepts necessary to understand the project's focus. We discuss general blockchain mechanisms, the inherent scalability challenges they face, and the Layer-2 solutions designed to overcome them, with a specific focus on Zero-Knowledge (ZK) Rollups.

1.1 Blockchain Fundamentals

1.1.1 Blockchain Structure

A blockchain is a distributed, append-only ledger that maintains a continuously growing list of records, called blocks, securely linked together using cryptography. Each block contains a cryptographic hash of the previous block, a timestamp, and transaction data [1]. This structure ensures data integrity and immutability.

A foundational component of this structure is the Merkle tree, which allows for efficient and secure verification of large data structures. By hashing pairs of nodes recursively until a single root hash is produced, a Merkle tree condenses vast amounts of transaction data into a single, succinct cryptographic commitment. This mechanism is extensively used in both Layer-1 blockchains (e.g., to summarize block transactions) and Layer-2 rollups (e.g., Sparse Merkle Trees to track state updates).

1.1.2 Consensus Protocols

Public, permissionless blockchains like Ethereum rely on decentralized consensus protocols (e.g., Proof of Work or Proof of Stake) to agree on the state of the ledger without a trusted central authority. While this decentralization provides strong security and censorship resistance, it imposes severe limitations on how quickly the network can process transactions.

1.1.3 Blockchain Types

While this project focuses on public, permissionless blockchains (specifically Ethereum), blockchains can also be permissioned or consortium-based. Public blockchains prioritize broad, decentralized participation at the cost of performance, whereas permissioned systems (e.g., Hyperledger Fabric) restrict access to known entities, allowing for more efficient, high-throughput consensus mechanisms like PBFT or Raft. However, for decentralized financial applications, the security guarantees of a public L1 like Ethereum are typically strictly required.

1.1.4 Scalability Metrics

When evaluating blockchain performance, several key metrics are considered:

- **Throughput (TPS):** The number of transactions the network can process per second.
- **Latency:** The time required for a transaction to be submitted, processed, and included in a block.
- **Finality (Finalization Time):** The time until a transaction is considered irreversible.
- **Cost per Transaction:** The economic cost (e.g., gas fees) paid by the user to execute the transaction.
- **Storage and DA Footprint:** The amount of data added to the ledger, impacting the storage requirements for full nodes.

1.1.5 The Blockchain Trilemma

The challenge of improving these metrics is often framed as the Blockchain Trilemma [2]. It suggests that a blockchain architecture can only achieve two out of three desirable properties:

1. **Decentralization:** The network is distributed across many nodes without central control.
2. **Security:** The network can resist attacks and manipulation.
3. **Scalability:** The network can handle a large volume of transactions at high speed and low cost.

1.1.6 The Blockchain Quadrilemma

Some recent literature expands the trilemma into a *quadrilemma* by introducing a fourth dimension: **Trust** or **Privacy**. In these models, attempting to scale the network via certain L2 solutions (like Validiums or sidechains) may compromise trustless operation, requiring users to place trust in external Data Availability committees or operators. ZK-Rollups aim to preserve the trustless nature of the base layer while significantly enhancing scalability, thereby providing a more balanced solution to the quadrilemma.

1.1.7 Ethereum Scalability Limitations

Ethereum L1 strictly prioritizes decentralization and security, resulting in inherently low throughput (approximately 15 TPS) and high latency. During periods of high demand, users bid up transaction fees, leading to severe economic friction [3]. Because every full node must sequentially process every transaction and store the resulting state, increasing the L1 block size or reducing block times to boost throughput would rapidly centralize the network by making node operation prohibitively expensive for average users.

Table 1.1: Ethereum vs High-throughput L1s

System	Design Target	Current Evidence	Why Relevant
Ethereum	Secure/decentralized settlement	24 TPS (Etherscan snapshot) [4]	Baseline L1 bott
Solana	High-throughput execution	1,045 TPS (Chainspect snapshot) [5]	Shows Ethereum fastest raw ex layer
Ethereum + Rollups	L1 settlement + L2 execution	Primary scaling technique [6]	Justifies rollup f

Ethereum L1 throughput is limited in practice because all nodes must independently verify the chain. Although a simple ETH transfer consumes only 21,000 gas [7], real Ethereum workloads (e.g., DeFi swaps, contract calls) are much heavier. Therefore, the observed TPS (24 TPS) is the right practical baseline, while theoretical simple-transfer TPS is only an upper-bound estimate [4, 7].

1.2 Layer-1 vs. Layer-2 Scaling

To address these limitations, scaling solutions are generally categorized into two layers.

Layer-1 (On-Chain) Scaling involves modifying the base blockchain protocol itself. Approaches include increasing block sizes, reducing block times, or implementing sharding (partitioning the database into smaller, faster parallel chains). However, these modifications risk compromising decentralization or security.

Layer-2 (Off-Chain) Scaling involves moving computation and state storage off the main chain while relying on the L1 for security and dispute resolution. The Ethereum community has largely adopted a "rollup-centric" scaling roadmap. Layer 2 (L2) systems extend Ethereum while relying on it for security [8]. According to Ethereum scaling documentation, L2 rollups have become the primary scaling technique [6]. Rollups execute off-chain and post data/state back to L1, retaining data on-chain while offloading computation [9].

Prominent L2 approaches include:

- **State Channels:** Allow users to transact off-chain multiple times and only submit the final state to the blockchain. They are fast but require participants to be online and are typically limited to specific applications.
- **Sidechains:** Independent blockchains that run parallel to the main chain, often with their own consensus mechanism. They are highly scalable but do not inherit the security of the main chain.
- **Plasma:** A framework that creates hierarchical tree structures of sidechains. While it inherits some security from L1 via fraud proofs, users must actively monitor the network, and data availability issues complicate mass withdrawals.
- **Optimistic Rollups:** Execute transactions off-chain and assume they are valid by default. They rely on "fraud proofs" where anyone can challenge an invalid transaction during a specific dispute window (typically 7 days). This leads to delayed finality for withdrawals.
- **ZK-Rollups:** Execute transactions off-chain and generate a cryptographic proof of validity. This proof is instantly verified by a smart contract on L1, ensuring deterministic finality without the need for a dispute window.

1.3 Rollups and ZK-Rollups

Rollups work by executing transactions off-chain in a separate environment. A specialized node, called a sequencer, groups many of these off-chain transactions into a single "batch." The sequencer then submits a summary of this batch (including the updated state root and necessary transaction data) to the L1 blockchain.

By grouping transactions, rollups amortize the fixed L1 submission and verification costs across the entire batch, dramatically increasing effective throughput and lowering the cost per transaction.

1.3.1 Zero-Knowledge Proofs

ZK-Rollups distinguish themselves from Optimistic Rollups by their verification mechanism. Instead of relying on a dispute time window and fraud proofs, ZK-Rollups utilize advanced cryptographic validity proofs—specifically Zero-Knowledge Proofs (ZKPs) such as zk-SNARKs or zk-STARKs [10].

The off-chain prover generates a mathematical proof attesting to the correctness of the batch’s state transition. A smart contract on L1 verifies this proof. Because verifying a ZKP is computationally much cheaper than re-executing thousands of transactions, L1 can guarantee the correctness of the L2 state instantly without executing the transactions itself. Modern implementations often utilize zkVMs (like RISC Zero) to execute arbitrary guest code and generate STARK proofs, which can subsequently be wrapped inside a SNARK (e.g., Groth16) for cheaper on-chain verification using Ethereum’s native pairing precompiles.

1.3.2 zk-SNARKs, zk-STARKs, and RISC Zero

- **zk-SNARKs** (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) are known for incredibly small proof sizes and fast, constant-time verification. However, they traditionally require a trusted setup phase, which introduces a potential security risk if the setup ceremony is compromised. Groth16 and PLONK are popular SNARK proving systems used in production rollups today.
- **zk-STARKs** (Zero-Knowledge Scalable Transparent Argument of Knowledge) require no trusted setup (relying on transparent cryptography like hash functions) and offer post-quantum security. However, they generally produce significantly larger proof sizes, which can increase the L1 gas cost required for verification [10].

A significant advancement in practical zero-knowledge proving is the emergence of zkVMs (Zero-Knowledge Virtual Machines), such as the RISC Zero zkVM. RISC Zero executes standard RISC-V bytecode and generates a STARK proof attesting to the correct execution of the code. Because STARK proofs are large and expensive to verify on-chain, they are often wrapped and compressed into a SNARK (like Groth16) using bilinear pairings before submission to Layer 1. This hybrid approach leverages

the transparent, scalable generation of STARKs while retaining the cheap, constant-size verification of SNARKs.

1.4 Data Availability (DA)

Rollup proofs verify computational correctness, but actual transaction data (or state diffs) is required to independently reconstruct and verify the rollup state. This concept is termed Data Availability (DA) [11]. For a rollup to be secure and trustless, the sequencer must publish this data so that anyone can run an independent node. If a rollup operator withholds data, users cannot verify the state or recover funds. It must be noted that DA is not simply 'metadata'—it is the core underlying transaction data necessary to derive the state transition.

Historically, ZK-Rollups posted this data to Ethereum L1 using standard transaction calldata. However, calldata is permanently stored on all Ethereum full nodes, making it prohibitively expensive and typically the dominant component of L2 transaction costs.

To mitigate this, Ethereum introduced EIP-4844 ("Proto-Danksharding"), which created a new transaction type that carries "blobs" of data [12]. Blobs are temporarily available on the consensus layer (typically for a few weeks) and are priced in a separate, significantly cheaper fee market. While the EVM execution layer cannot read the blob data directly, it can verify cryptographic commitments (such as KZG commitments) to the blob data [12]. This fundamental shift significantly alters the cost structure of DA for rollups [11, 13]. Off-chain DA solutions offer even cheaper alternatives but introduce different trust assumptions, pushing the architecture toward a Validium model rather than a pure rollup.

1.5 Motivation

Blockchain technology has emerged as a transformative innovation, providing a secure, distributed ledger that enables trust among untrusted entities without relying on centralized authorities or intermediaries. Its defining properties, including immutability, transparency, auditability, autonomy, and resilience, position blockchain as one of the most disruptive digital technologies of the past decade [14].

Initially conceived as the foundation of Bitcoin [1], blockchain's applications have rapidly expanded beyond cryptocurrencies. Ethereum [3] introduced programmability through smart contracts, enabling decentralized applications (dApps) that support diverse use cases. Today, blockchain is being deployed in finance, insurance, supply

chain management, healthcare, identity management, registry services, stock trading, the Internet of Things (IoT), and energy systems.

Blockchain adoption has grown exponentially across various sectors. However, as the user base and transaction volumes expand, the underlying networks face significant scalability challenges. Ethereum Layer-1, a leading smart contract platform, provides strong security and decentralization but remains constrained by limited throughput, network congestion, and severe fee volatility. Under typical conditions, Ethereum’s Layer-1 processes approximately 15–25 transactions per second (TPS), far below the thousands of TPS required for global-scale adoption. Its sequential execution model and consensus mechanism traditionally limit the network capacity.

This fundamental limitation reflects the widely recognized blockchain trilemma—which posits that decentralized systems must balance scalability, security, and decentralization, and cannot maximize all simultaneously. Consequently, a robust scaling strategy is required to achieve high transaction throughput and low latency without compromising the security guarantees of the base layer.

Ethereum’s current scaling direction relies heavily on Layer-2 (L2) rollups. Rollups move transaction execution off-chain, grouping many off-chain transactions into large batches, while continuing to use Layer-1 (L1) for settlement, verification, and data availability. This approach amortizes the fixed L1 submission costs across multiple transactions, effectively increasing throughput while retaining Ethereum as the ultimate security layer.

Within this rollup-centric design space, Zero-Knowledge (ZK) Rollups are particularly attractive. They bundle hundreds or thousands of off-chain transfers into a single transaction and submit cryptographic validity proofs (such as zk-SNARKs or zk-STARKs) that allow Layer-1 smart contracts to mathematically verify off-chain state transitions. This avoids the need for L1 to re-execute every transaction, preserving the base layer’s security model while significantly reducing the computational burden on L1.

1.6 Problem Statement

Ethereum Layer-1 is constrained by limited throughput (historically processing 15-25 TPS) and high fee volatility [3]. Zero-knowledge rollups (ZK rollups) have emerged as a promising Layer-2 (L2) solution to address these scalability issues. By processing thousands of transactions off-chain and submitting compact validity proofs to L1, ZK rollups can theoretically achieve significantly higher throughput while inheriting Ethereum’s robust security model [15].

However, real-world deployments of ZK rollups reveal critical bottlenecks that limit their practical effectiveness. Practical ZK-Rollup performance is dictated by several interacting and often competing factors. These include batch formation limits, sequencer scheduling algorithms, data availability footprints, Layer-1 submission costs, and compute-intensive proof-generation overhead [9, 16]. Importantly, these factors do not affect performance independently. For instance, increasing the maximum batch size reduces the amortized Layer-1 data cost per transaction but simultaneously increases the queueing delay (latency) for early transactions waiting in the pool for the batch to seal.

Currently, observing these trade-offs requires either complex theoretical modeling or passive analysis of production mainnet systems (like zkSync Era or Starknet). Production systems are highly optimized for reliability, full EVM compatibility, and end-user security rather than for scientific experimentation. They are difficult to modify, and their real-world workloads cannot be perfectly replicated to test different configurations under identical conditions.

Therefore, the core research problem addressed in this report is the lack of a compact, modular, and reproducible experimental framework designed specifically to isolate how batching, sequencing, proof-delay, and data availability choices affect ZK-Rollup throughput, latency, finalization time, and cost behavior under strictly comparable, controlled synthetic workloads.

1.7 Research Questions

To guide this investigation, the project will answer the following research questions:

1. **RQ1:** What is the quantitative impact of batch size on throughput, latency, proof-generation overhead, and per-transaction cost?
2. **RQ2:** How do sequencer scheduling policies affect transaction ordering, fairness, latency, and execution correctness?
3. **RQ3:** What are the cost implications of calldata, EIP-4844 blob DA, and offchain DA modes?
4. **RQ4:** What practical bottlenecks emerge when implementing a modular research ZK-rollup prototype?

These questions are designed to yield empirical, reproducible evidence to guide the design and optimization of ZK-Rollup systems.

1.8 Research Objectives

To systematically answer the research questions, the project defines the following specific objectives emphasizing modularity and reproducibility:

- **Develop a Modular Prototype:** Design and implement a modular ZK-Rollup prototype framework deployable on a local Ethereum testnet, featuring distinct Sequencer, Executor, Submitter, and Smart Contract components.
- **Establish Benchmarking Infrastructure:** Create a synthetic Poisson workload generator and an automated orchestration pipeline to execute reproducible configuration sweeps.
- **Investigate Batching Dynamics:** Execute experiments to benchmark baseline performance metrics and measure the impact of varying batch sizes on latency and L1 gas costs.
- **Evaluate Sequencer Policies:** Implement and compare scheduling algorithms to analyze their impact on fairness, latency, and execution correctness.
- **Analyze Data Availability:** Support DA mode comparison by actively toggling between calldata, EIP-4844 blobs, and offchain DA modes to measure L1 footprint and cost differences.
- **Isolate Prover Overhead:** Utilize benchmark-mode proof-delay mock provers and basic risc0 proof integration to isolate prover-stage effects from L1 submission overhead and observe step-function scaling behavior.
- **Analyze Trade-offs:** Synthesize the collected metrics (throughput, latency, cost, DA) into quantitative trade-off analyses (e.g., latency vs cost).

1.8.1 Mapping Research Objectives to Evaluation Metrics

The achievement of these objectives is measured through specific performance metrics collected during the benchmarking phases. The mapping between research questions, objectives, implemented components, and evaluation metrics is carefully structured to ensure empirical traceability.

- **Throughput and Latency:** Measured to answer RQ1 and RQ2, these metrics are derived from the Workload Generator and Sequencer telemetry, focusing on transaction queue wait times and batch processing rates.

- **Per-Transaction Cost:** Extracted from the L1 smart contracts to address RQ1 and RQ3, this metric calculates the amortized gas cost per transaction based on batch size and DA mode.
- **Proof Generation Overhead:** Tracked within the Executor module to address RQ1 and RQ4, this metric measures wall-clock time and RISC0 cycles required for cryptographic proof synthesis.
- **Execution Correctness and Fairness:** Evaluated through the Sequencer’s scheduling logs and the Executor’s state transition success rates, addressing RQ2 by observing how global sorting impacts nonce ordering.

1.9 Project Scope and Assumptions

Scope: The project is strictly scoped to building a research prototype. Complex cryptographic proof generation is primarily modeled via a mock delay, and execution is limited to simplified transfer state transitions.

Assumptions: We assume that the local dev-chain (Hardhat) environment accurately reflects the relative (though not absolute) L1 execution costs of mainnet. We also assume that the synthetic Poisson workload provides a sufficient baseline to measure relative queueing dynamics, acknowledging that real-world DeFi traffic is significantly more bursty.

1.9.1 Research Question Mapping

Table 1.2 maps the research questions to the objectives, components, and report sections.

Table 1.2: Research Question Mapping

RQ	Related Objective	Related Component	Related Metric	Report Section
RQ1	Design modular architecture	All components	Architecture validation	Ch. ??, 4
RQ2	Evaluate batch/DA modes	Sequencer, Submitter	TPS, Latency, Cost, DA Footprint	Ch. ??, ??
RQ3	Isolate prover effects	Executor, Mock Prover	Proof Delay, Finalization Time	Ch. ??, ??
RQ4	Identify bottlenecks	Integration Pipeline	Development lessons	Ch. 4, ??

1.10 Contributions

The primary contributions of this project include:

1. The design and implementation of a modular ZK-rollup benchmarking prototype that strictly separates sequencing, execution, proving, and submission stages into distinct Rust microservices.
2. A controlled experimental framework, including a Python-based Poisson workload generator, capable of executing reproducible configuration sweeps to measure throughput, latency, finalization time, and cost metrics.
3. Empirical analysis identifying the critical throughput–latency–cost trade-offs across different batching parameters and Data Availability (DA) modes based on extensive local testnet benchmarking.
4. The demonstration of both a benchmark-mode parameterized proof delay and a real `risc0` proof integration as a valid methodology for analyzing system-level queueing dynamics and fixed setup costs independently of full production zkEVM complexity.

1.11 Document Structure

The remainder of this report is structured as follows. Chapter ?? provides a detailed background on blockchain scalability concepts, the scalability trilemma, and ZK-Rollups. Chapter 2 reviews relevant literature across broad scalability taxonomies and focused ZK-rollup bottlenecks. Chapter ?? outlines the refined problem statement, research questions, and objectives. Chapter ?? presents the high-level system architecture and component interactions. Chapter 3 discusses the research methodology and evaluation metrics. Chapter 4 delves deeply into the actual implementation of the prototype’s core components (Sequencer, Executor, Submitter, and Smart Contracts). Chapter ?? outlines the containerized testing environment and benchmark configurations, followed by Chapter ?? which presents the findings. Chapter ?? provides an in-depth analysis of the throughput–latency–cost trade-offs observed during benchmarking. Chapter ?? addresses potential threats to the validity of the results. Chapter ?? documents project management and the team’s individual contributions. Finally, Chapter 6 concludes the report and suggests directions for future work.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the existing literature surrounding blockchain scalability, categorized into broad scalability taxonomies and focused research on ZK-Rollup bottlenecks. It concludes with a synthesis that positions this project within the current research landscape.

2.1 Broad Scalability Taxonomy

Research into blockchain scalability generally falls into three broad categories: write-performance, read-performance, and storage scalability.

2.1.1 Write-Performance Scalability

Write-performance refers to the network’s ability to process and confirm transactions (Throughput and Latency). Existing literature extensively explores:

- **On-chain (Layer-1) scaling:** Modifying consensus mechanisms (e.g., transitioning from Proof-of-Work to Proof-of-Stake or adopting DAG structures) and sharding the network state [17].
- **Off-chain (Layer-2) scaling:** Moving execution to state channels, sidechains, or rollups. Rollups, in particular, are favored because they do not require a separate consensus mechanism for security, relying instead on the L1 [15].

2.1.2 Read-Performance Scalability

As the blockchain grows append-only, the sheer volume of data makes it difficult for lightweight nodes to query the state (Read-Performance). ZK-Rollups inadvertently exacerbate the read-performance issue on L2. To reconstruct the L2 state, nodes must parse massive amounts of compressed transaction data or state diffs posted to L1, a process that is resource-intensive.

2.1.3 Storage Scalability

Storage Scalability deals with the massive disk space required for full nodes [14]. While state expiry and stateless clients are being researched for L1 storage scaling, L2 solutions try to minimize the on-chain footprint by compressing transaction data or using commitments rather than raw state values.

2.2 ZK-Rollup Bottleneck Research

With the industry shift towards rollups, research has increasingly focused on the specific bottlenecks within the L2 pipeline.

2.2.1 Sequencer Policies and Batching

The sequencer is the central orchestrator of a rollup. Motepalli et al. [18] discuss the implications of centralized sequencers and various transaction ordering policies (e.g., First-Come-First-Served vs. Priority-Fee). The literature highlights that the batch formation strategy directly dictates the trade-off between user latency (waiting for a batch to seal) and L1 cost amortization (waiting for more transactions to share the L1 fee).

2.2.2 Prover Overhead

Generating cryptographic proofs is notoriously resource-intensive. Habib et al. [16] and Chaliasos et al. [9, 19] analyze the significant computational overhead introduced by the proving stage. While production systems utilize massive hardware acceleration (GPUs/FPGAs) and recursive proof aggregation, the proving time remains a major contributor to the delay in achieving L1 hard finality.

2.2.3 Data Availability (DA) Dynamics

The cost of posting data to L1 has historically dominated rollup transaction fees. Saif et al. [11] and Lavour et al. [13] analyze the impact of EIP-4844 blobs. Their work demonstrates that transitioning from standard calldata to blobs significantly reduces DA costs, though it introduces new dynamics regarding blob pricing and temporary data availability windows. Production ZK systems (e.g., Polygon zkEVM, Scroll, zkSync) use very different hardware setups and prover designs. For instance, Polygon zkEVM demands high-capacity setups (e.g., 96-core CPU, 768 GB RAM) [9].

This diversity makes identical-hardware comparisons difficult, justifying the use of a research-grade prototype with parameterized proof delays.

2.2.4 Summary of Existing Literature

Table 2.1 summarizes the key literature reviewed, highlighting the focus areas, methodologies, key findings, limitations, and relevance to this project.

Table 2.1: Comparison of Key Literature on Blockchain Scalability and ZK-Rollups

Paper / Authors	Focus	Methodology	Key Findings	Limitations	Relevance to this Project
Chaliasos et al. [9]	ZK-Rollup Benchmarking	Empirical analysis of public ZK-rollup datasets.	Highlighted L1 DA costs and L2 proving times as major bottlenecks.	Relies on diverse production systems, making apples-to-apples comparisons difficult.	Provides the foundation for the cost models and evaluation metrics used in our controlled framework.
Saif et al. [11]	Data Availability in L2s	Survey of DA mechanisms across various rollups.	Identified the shift from calldata to EIP-4844 blobs as a crucial cost-saving measure.	Does not provide empirical execution metrics under varying load.	Motivates our empirical evaluation of Calldata vs. Blob vs. Offchain DA modes.
EIP-4844 [12]	Ethereum Layer-1 Updates	Specification for Proto-Danksharding.	Introduces blob-carrying transactions to drastically lower DA costs.	Theoretical specification; practical L2 integration requires custom sequencer logic.	Informs the implementation of the Blob DA mode and Blob-Packing policy.
Sanka et al. [14]	General Scalability	Systematic literature review of blockchain scaling.	Classifies scaling solutions into L1 and L2, emphasizing the trilemma.	Broad survey; lacks deep technical analysis of ZK proving bottlenecks.	Establishes the broad background and motivation for L2 research.

2.3 Rollup Benchmarking and Public Datasets

Benchmarking ZK-rollups is multifaceted due to complex deployments and differing design choices. Chaliasos et al. [9] emphasize that creating standardized benchmarks for ZK-rollups is challenging because varying levels of ZK-EVM equivalence make common payloads hard to define. They provide analytical cost models demonstrating that per-transaction cost is a function of fixed batch settlement costs divided by batch size, plus marginal execution and data costs.

Additionally, recent studies analyze public datasets from deployed systems. For example, Silva et al. [20] provide comprehensive analyses of zkSync Era traces. While these studies offer valuable, empirical insight into real-world rollup behavior, they are less suited for isolating the causal impact of individual configuration choices. In a production environment, it is practically impossible for a researcher to independently

modify the sequencer’s internal batching rules, scheduling algorithms, or DA modes under strictly identical, controlled workloads. Thus, a gap remains for a simplified, controlled benchmarking prototype that trades full EVM capabilities for the ability to strictly isolate and measure specific variables, such as how parameterizing prover delays impacts system-wide queueing.

2.4 Synthesis and Positioning of This Work

Table 2.2 summarizes the identified research areas, existing limitations, and how this project addresses them.

Table 2.2: Synthesis of Literature and Project Positioning

Research Area	Existing Work / Focus	Limitation in Literature	How This Project Responds
Write Scalability	Broad surveys on L1/L2 scaling [15, 17]	Often theoretical; lacks empirical implementation data.	Implements a functioning L2 prototype framework.
Prover Bottlenecks	Analysis of ZKP generation costs [16]	Requires massive hardware to test real prover limits.	Uses benchmark-mode mock proving to isolate systems-level queueing, alongside basic real proof integration.
DA Costs	Analysis of EIP-4844 blobs vs calldata [11]	Usually observed passively on mainnet.	Allows active configuration toggling between calldata and blobs for direct L1 gas comparison.
Rollup Benchmarking	Benchmarking framework analysis [9]	Production ZK-EVMs are too complex to easily modify core sequencing logic.	Provides a controlled, synthetic benchmarking framework for direct configuration comparisons.

While prior work provides insight into deployed rollups, this project focuses on controlled, configuration-level experimentation. Most benchmarking studies [9] passively observe public production networks like Starknet or zkSync. Because these production systems prioritize reliability and full EVM compatibility, their core sequencing logic and DA selection cannot be easily toggled for controlled A/B testing.

In contrast, our prototype prioritizes controlled experimentation, modularity, and observability. This approach provides an environment to conduct repeatable, isolated studies to observe exactly how individual design choices—such as batch size limits, priority scheduling heuristics, or DA mode selection—shift fundamental throughput–latency–cost behavior without the noise of production mainnet traffic. By addressing this specific gap, the project contributes clear, empirical evidence on the trade-offs of modular ZK-rollup design.

CHAPTER 3

METHODOLOGY / SYSTEM DESIGN

This chapter outlines the research design, the modeling of workloads and configurations, and the experimental procedures used to answer the research questions.

3.1 High-Level Architecture Overview

The prototype developed for this project separates the standard ZK-Rollup pipeline into distinct, containerized microservices. This modular architecture follows Domain-Driven Design (DDD) principles and is explicitly designed to support observability, rapid configuration changes, and controlled benchmarking rather than functioning as a secure production deployment.

The system’s end-to-end execution flow is strictly defined as:

Workload Generator → Sequencer → Executor (gRPC) → Submitter →
L1 Contracts

This pipeline processes telemetry data by consuming raw JSON/CSV metrics from each component, allowing a Data Tools analytics pipeline to aggregate and analyze the metrics. Local environment execution is orchestrated via `docker compose` utilizing specific profiles to boot the core stack, the benchmark generator, and the reporting tools. By decoupling the transaction lifecycle—validation, batching, execution/proving, and submission—the prototype enables isolating the specific variables (like batch size or DA mode) required to answer the research questions.

3.1.1 Prototype vs. Production Rollup Comparison

It is critical to distinguish this *research prototype* from a *production rollup*. Production rollups (like zkSync Era or Polygon zkEVM) require full Ethereum Virtual Machine (EVM) compatibility, complex fraud/validity proof mechanisms handling massive state trees, and decentralized sequencer networks designed to prevent censorship.

They operate in a Byzantine environment requiring strong liveness and safety guarantees.

In contrast, our prototype operates a *transfer-centric research logic*. It does not implement a full zkEVM. Instead, it implements a simplified state-transition engine tailored for transfer-style workloads and Merkle state updates. Furthermore, the prototype operates in a controlled, non-Byzantine local Hardhat environment. Data Availability (DA) supports calldata and EIP-4844 blobs natively via the L1 node, but off-chain DA is implemented only as a local simulated stub, not a real integration with decentralized networks like IPFS or Celestia. This focused scope allows for precise measurement of specific configuration trade-offs—such as batch size limits or DA selection—without the massive engineering overhead of a production-grade system.

3.2 Transaction Lifecycle and Data Flow

The transaction lifecycle spans both the data plane (handling transaction execution) and the control plane (handling orchestration and metric collection). The end-to-end flow is detailed below.

1. **Ingestion (Workload Generator):** A Python-based synthetic Poisson Workload Generator synthesizes transaction payloads (primarily ERC-20 style transfers) and submits them to the Sequencer.
2. **Sequencing and Batching (Sequencer):** The Sequencer, implemented in Rust, validates the transactions and holds them in a local transaction pool. It orders them according to the configured scheduling policy (e.g., FCFS, FeePriority, or TimeBoost). A background Batching Engine evaluates triggers (such as maximum batch size or a timeout interval). Once a trigger condition is met, the batch is "sealed" and dispatched. This phase establishes *soft finality*, providing immediate confirmation to users that their transaction is scheduled.
3. **Execution (Executor):** The Executor, also written in Rust, receives the sealed batch via a gRPC stream. It applies the transactions against its local canonical state representation (updating account balances and nonces via a Sparse Merkle Tree) and computes the new global state root.
4. **Proving (Prover Subsystem):** The Executor delegates proof generation to the Prover. For high-volume benchmarking, a *mock prover* introduces parameterized delays, simulating computational overhead without bottlenecking on hardware.

For cycle-analysis experiments, a real *RISC0 prover* backend is utilized, executing the state transitions within the zkVM to generate authentic STARK proofs, which are subsequently compressed into Groth16 proofs.

5. **Submission (Submitter):** The Submitter (Rust) acts as the bridge between L2 and L1. It listens to the Executor’s gRPC stream, packages the execution results, the proof, and the transaction data into a final payload. Crucially, the Submitter’s Data Availability Formatter structures the data according to the selected DA mode (e.g., standard calldata, EIP-4844 blobs, or off-chain stubs) and manages the retry logic via a Saga Workflow.
6. **L1 Settlement (Smart Contracts):** The Submitter broadcasts the transaction to the L1 Smart Contracts running on a local Hardhat node. The L1 contract verifies the DA commitments and the proof using either a Mock Verifier (for benchmark mode) or a real Groth16 Verifier (for RISC0 mode), subsequently updating the on-chain canonical state root. This finalizes the transaction, establishing *hard finality*.

Throughout this lifecycle, a metrics pipeline continuously logs telemetry data. This data is later processed by the Data Tools (e.g., `aggregate.py` and `stats.py`) to generate the performance analyses presented in this report.

3.3 Component Interactions

The system components interact synchronously across well-defined boundaries. Table 3.1 summarizes the language and core responsibility of each component in the pipeline.

Table 3.1: Component Responsibility Summary

Component	Language	Core Responsibility
Workload Gen.	Python	Synthesize controlled traffic (Poisson arrivals) and simulate varying network load.
Sequencer	Rust	Validate, order, and batch L2 transactions using policies like FCFS; enforce timeout triggers.
Executor	Rust	Receive gRPC batches, execute a transfer-centric state transition over a RocksDB Sparse Merkle Tree, and invoke the Prover.
Prover	Rust	A subsystem of the Executor using <code>risc0_prover</code> (or mock delays) to generate cryptographic validity artifacts.
Submitter	Rust	Handle L1 Ethereum Adapter interactions, manage DA formatting (Calldata/Blobs), and ensure reliable transaction broadcast via a Saga loop.
Contracts	Solidity	L1 bridge; validates DA commitments, verifies proofs via <code>MockVerifier</code> or <code>Groth16</code> , and finalizes the batch.

The interactions ensure a strict cryptographic link. The Sequencer’s transaction handling is isolated from batch creation. When a batch is sealed, it is sent to the Executor via gRPC. The Executor maintains a trace of the execution and invokes the prover backend. The resulting enriched payload (DA commitment and proof) is broadcast via a stream to the Submitter, which handles the final L1 gas estimation and submission.

3.4 Data Flow and Control Flow

Data flows strictly unidirectionally from the Sequencer through the Executor to the Submitter via highly typed gRPC channels. Control flow is handled asynchronously within each component; for example, the Submitter’s Saga engine operates independently of the Sequencer’s batch generation loops, allowing the system to handle L1 RPC instability without halting L2 ingestion.

3.5 Design Rationale

The prototype utilizes several established software design patterns to maintain modularity:

- **Strategy Pattern:** Used in the Sequencer to allow hot-swapping of different transaction scheduling policies without altering the core batching logic.

- **Pipeline Pattern:** Employed in the Executor and Submitter to process batches through discrete validation, execution, and formatting stages.
- **Saga Workflow Engine:** The Submitter implements a robust Saga pattern backed by a local database (SQLite/Postgres) to handle reliable retries, nonce management, and idempotency when interacting with the volatile L1 RPC.
- **Observer Pattern:** Used by the Sequencer’s L1 Event Listener to monitor the bridge contract for forced transactions (e.g., L1 to L2 deposits).

3.6 Prototype vs. Production Distinctions

To contextualize the architecture, Table 3.2 outlines the key differences between this research prototype and a production mainnet rollout.

Table 3.2: Prototype vs. Production Rollup Architecture

Feature		Production Rollup (e.g., zkSync Era)	This Research Prototype
Execution Environment		Full zkEVM, capable of running arbitrary Solidity smart contracts.	Simplified state-transition engine tailored for transfer-style workloads.
Proving System		Massively parallelized, hardware-accelerated recursive proving (e.g., Boojum/PLONK).	Parameterized mock-delay or basic RISC0 host for bottleneck isolation.
L1 Network		Deployed on Ethereum Mainnet.	Benchmarked against a local, controlled Hardhat node to eliminate external network jitter.
Data Availability		Often integrates with external DA layers (e.g., Celestia) alongside L1.	Focuses strictly on L1 DA (Call-data and EIP-4844 blobs); off-chain DA is a simulated stub.

3.7 Research Design

The research employs a quantitative experimental design built upon the developed modular ZK-rollup prototype. By using a controlled benchmarking framework rather than a live network, we can strictly isolate independent variables (such as batch size or DA mode) while maintaining identical dependent variables (the synthesized workload traffic).

3.8 Workload Model

To isolate the effects of system configurations (such as batch size or sequencer time-outs) from external noise, the benchmarking methodology relies entirely on controlled, synthetic workloads. It explicitly does not use trace-driven replays of mainnet traffic. This approach minimizes external confounders, enabling clear attribution of performance changes to specific configuration adjustments.

A Python-based `poisson_generator.py` script synthesizes transaction requests using a seeded Poisson arrival process to simulate realistic, steady-state user traffic. The generator allows parameterized control over the offered input rate (TPS) and transaction mixes. However, it lacks implementation for highly bursty, volatile transfer patterns typical of extreme DeFi events. Instead, burst tests are simulated using programmatic rate stepping (e.g., maintaining an 8 TPS base rate before injecting an 80 TPS burst).

3.8.1 Why a Local Hardhat Node and Mock Prover Are Used

A fundamental methodological choice was utilizing a local Hardhat node combined with a Mock Prover for the majority of the benchmarking sweeps (Stages 1 through 4), rather than executing on public testnets with real hardware-intensive provers.

- **Local Hardhat Node:** Running the L1 settlement layer on a local Hardhat node (configured for automatic or ultra-fast mining, e.g., 12 seconds per block) eliminates network congestion, fluctuating base fees, and unpredictable L1 finality variance. This ensures that the L1 submission phase behaves deterministically, allowing the metrics collection to focus exclusively on L2 sequencing and batching dynamics.
- **Mock Prover:** True cryptographic proof generation (e.g., using RISC Zero) is an extremely compute-bound process that acts as a massive bottleneck, often masking subtle scheduling and queueing dynamics. By using a Mock Prover with a parameterized delay, the framework simulates proof generation overhead without requiring a specialized GPU cluster. Real RISC Zero proving is deliberately separated into isolated Stage 5 experiments to profile cycle behavior and step-function limits independently.

To ensure the load generator can saturate the sequencer without becoming a bottleneck itself, it relies on unsigned or pre-signed user transactions when `ALLOW_UNSIGNED_USER_TXS` is enabled in the benchmarking stack.

3.9 Configuration Spaces

3.9.1 Batching Policy Space

The sequencer evaluates batch triggers continuously. The primary variables are `max_batch_size` (triggering a seal when N transactions accumulate) and `timeout_interval_ms` (triggering a seal when time T elapses).

3.9.2 Sequencing Policy Space

The sequencer supports hot-swapping scheduling policies via the Strategy pattern. While FCFS (First-Come-First-Served) is the default baseline, the methodology supports evaluating Fee-Priority policies to observe latency changes during high load.

3.9.3 Data Availability Modes

The methodology dictates testing two distinct L1 DA submission modes:

- **Calldata:** Compressing the transaction payload and sending it as standard L1 transaction data.
- **EIP-4844 Blobs:** Formatting the payload into a blob and submitting a blob-carrying transaction, significantly reducing the EVM execution gas cost.

3.10 Analytical Cost Model

The empirical benchmarking uses static configurations swept explicitly to gather data on cost behavior. While theoretical "dynamic batching cost models" (which algorithmically predict optimal batch sizes based on fluctuating L1 base fees) are discussed in early project proposals, they are analytical constructs and are explicitly not implemented in the active runtime codebase.

Following the methodology of Chaliasos et al. [9], the cost model utilized for analyzing results splits into fixed batch costs (L1 settlement base fees, proof verification) and marginal transaction costs (DA bytes, witness generation, execution).

Table 3.3: ZK-Rollup Cost Model [9, 11]

Cost Category	Examples	Fixed/Marginal	Source
L1 Settlement	Commit batch, base fee	Fixed per batch	Chaliasos [9]
Proof Verification	Groth16 pairing check	Fixed per batch	Chaliasos [9]
Data Availability (DA)	calldata, blob bytes	Marginal / payload-dependent	Saif [11]
Proving Time	witness + proof generation	Marginal / cycles	Chaliasos [9]

The fundamental equation governing Layer-1 gas cost per transaction is:

$$\text{Cost per tx} = \left(\frac{\text{fixed batch settlement \& proof cost}}{\text{batch size}} \right) + \text{marginal DA cost per tx}$$

Larger batches reduce the amortized fixed cost per transaction but inherently increase queueing latency.

It is important to note that the USD cost metrics reported in this document are derived from this gas model combined with assumed static environment parameters (e.g., `ETH_PRICE_USD` = \$2500, `REGULAR_GAS_PRICE_GWEI` = 2.0). However, due to L1 base fee fluctuations, gas units (e.g., L1 Gas Used per batch) remain the only stable, comparable metric for cross-configuration efficiency evaluations.

3.11 Evaluation Metrics

Telemetry data is collected as raw JSON/CSV metrics in the `benchmark-suite/metrics/` directory. The Data Tools analytics pipeline subsequently processes this data using Python aggregation scripts to compute the final metrics.

Table 3.4: Key Benchmarking Metrics

Metric	Definition	Unit	Why it matters
Throughput	Rate of successful transaction inclusion	TPS	Indicates capacity limits of the system
Latency	Time from transaction submission to soft finality	ms	Affects user experience
Finalization	Time from batch sealing to hard finality on L1	ms	Affects system security and dispute windows
Gas Cost	L1 execution cost per transaction or per batch	Wei	Determines the economic viability of the rollup
DA Footprint	Size of the compressed data posted to L1	Bytes	Directly correlates with calldata costs

Key metrics include Committed TPS (transactions committed to L1 per second), average and P95 latency, proof delay (seconds per proof), DA bytes posted, cost per transaction, and batch finalization time [9]. As dictated by the benchmarking plan, statistical analysis must report mean, median, standard deviation, 95% confidence intervals, and latency percentiles (P50, P90, P95, P99) to accurately reflect the skew often found in distributed system queueing networks.

3.12 Experimental Procedure and Hypotheses

Experiments are conducted by methodically sweeping the configuration variables via bash orchestration scripts.

- **Batch Size Sweep:** We hypothesize that larger batches will decrease cost per transaction but increase average latency.
- **DA Mode Comparison:** We hypothesize that blob mode will show a drastically reduced L1 gas footprint compared to calldata.
- **Mock Proof Delay Sweep:** Utilizing parameterized proof delays, we hypothesize that simulating complex proof generation will shift the system bottleneck from the L1 submission rate to the Executor/Prover queue.

CHAPTER 4

IMPLEMENTATION

4.1 Implementation Overview

This chapter details the architecture and implementation of the modular ZK-rollup benchmarking prototype developed for this research project. The system separates sequencing, execution, proving, and submission stages to allow controlled configuration sweeps over specific bottlenecks. The core components are implemented in Rust using Domain-Driven Design (DDD) principles and SOLID architectural patterns. The system relies on containerized microservices (via Docker Compose) for orchestrated benchmarking.

Table 4.1: Implementation Status Table

Feature	Status	Limitation
Sequencer	Implemented	Basic state root tracking
Executor	Implemented	Transfer-centric research logic
Prover	Implemented (RISC0/Mock)	Mock/Delay mode used for benchmarks to isolate c
Submitter	Implemented	Relies on local dev-chain (Hardhat)
Workload Gen	Implemented (Python)	Steady-state synthetic (Poisson)

4.2 Workload Generator

The Workload Generator is responsible for synthesizing a realistic arrival stream of Layer-2 transactions. It drives the synthetic load for all experiments in the benchmark suite, ensuring that each configuration is tested under identical traffic conditions.

- **Language:** Python.
- **Generation Method:** It employs an Arrival Process Engine configured to generate steady-state traffic following a Poisson distribution. This accurately models independent user arrivals. Note that the generator currently focuses on steady-

state and skewed transaction mixes but does not implement complex bursty transfer patterns.

- **Transaction Format:** The generator synthesizes JSON-RPC transaction requests. To ensure validity during high-throughput benchmarks without requiring expensive, compute-bound cryptographic signing operations from the load generator itself, the system uses a dummy signature approach. Specifically, the sequencer's `UserTransaction::hash()` function deliberately excludes `calldata` and `tx_type` to allow these dummy signatures to pass verification.
- **Input/Output Files:** The generator interacts with the sequencer via an HTTP/RPC client. It tags transactions with Experiment IDs and Run IDs and outputs detailed telemetry, specifically `workload.<exp_id>.json` and `tx_log.<run_id>.csv`, which record submission timestamps and latency.

4.3 Sequencer

The Sequencer is the entry point for L2 transactions. It is responsible for ingesting, validating, and grouping transactions into batches before forwarding them to the execution stage. Strict architectural rules dictate that the sequencer interacts with its database via a defined Storage trait and keeps business logic separate from the `src/bin/` entry point.

- **Language:** Rust.
- **API Interface:** Exposes a JSON-RPC Server to receive transactions from users and the Python Workload Generator.
- **Transaction Flow (Validity Checker & Transaction Pool):** Incoming transactions pass through a Validity Checker that interfaces with a pessimistic state cache (verifying nonces and balances locally). Valid transactions are then pushed to the Transaction Pool, an in-memory queue awaiting selection.
- **Batch Formation (Batching Engine):** A background Batching Engine continuously polls the transaction pool to form batches based on explicitly configured triggers. The primary sealing triggers are reaching a size threshold (`max_batch_size`) or a timeout threshold (`timeout_interval_ms`). Adaptive batching logic was also implemented but identified as fundamentally flawed during testing due to hysteresis threshold traps (as discussed in the Results chapter).

- **Scheduling Policies:** The sequencer uses the Strategy pattern to support different transaction ordering heuristics:
 - *FCFS*: Strict temporal arrival order.
 - *FeePriority*: Sorted by gas price to maximize sequencer revenue.
 - *TimeBoost*: Time-windowed grouping to balance fee revenue and latency fairness.
 - *FairBFT*: Timestamp-ordered execution.
 - *BlobPacking*: Specifically optimizes for EIP-4844 by calculating a multi-dimensional score (`feeScore`, `fitScore`, `waitScore`) to maximize blob utilization.
- **Output:** Completed batches are dispatched to the Executor via a gRPC stream. Configuration is managed via TOML files (e.g., `sequencer.template.toml`), which are processed via `envsubst` during container startup (notably requiring explicit bash environment variables since default fallbacks are unsupported).

4.4 Executor

The Executor serves as the host-side state transition and proving coordinator.

- **Language:** Rust (requires `nightly-2024-08-01` toolchain due to `risc0` dependencies).
- **Execution Role (State Transition Function):** It processes a `publish_batch` request via gRPC. It parses and normalizes transactions, and executes the state transition function (STF) using a `SimpleTransactionEngine`. The STF strictly verifies account balances and strict nonce ordering. If a transaction nonce is out of order, the STF rejects the transaction, preventing subsequent execution for that account.
- **State Manager:** State updates are persisted and tracked using a Sparse Merkle Tree backed by a `RocksDbStateManager`. This structure generates the exact Merkle proofs required for the zero-knowledge circuits.
- **Prover Integration:** After execution, it builds an `ExecutionTraceV1` and invokes the configured prover backend to generate proof artifacts.
- **Output:** It emits an enriched batch payload (containing the DA commitment and proof) back to the Submitter stream and logs execution metrics.

- **Limitations:** The executor is a simplified state-transition engine tailored for transfer-style workloads and Merkle state updates (transfer-centric research logic). It is explicitly not a full EVM or zkEVM.

4.5 Prover and Benchmark-Mode Delay

To allow for scalability experiments without being heavily constrained by the massive hardware requirements of actual cryptographic proof generation, the prototype relies heavily on a mock proving path for analytical benchmarking, while still supporting a base real prover.

- **Mock Prover Mode:** During standard benchmark execution, the system uses a parameterized time delay to simulate cryptographic overhead. This allows the study to isolate systems-level bottlenecks (sequencer queue growth, database I/O, network latency) from compute-bound cryptographic constraints.
- **Real Prover Backend:** A `risc0_prover` implementation exists and is used as the real proof path backend for the executor (`PROVER_BACKEND=risc0`). When enabled via `REQUIRE_REAL_PROOFS`, it executes the guest code inside the RISC0 zkVM, generating STARK proofs that can be wrapped in Groth16. It correctly processes the Merkle state updates, exhibiting step-function latency at segment boundaries with a ~ 70 -80s fixed overhead, and produces constant 256-byte proofs.
- **Deprecation of Initial Proposals:** Early progress reports mentioned parallelized proof generation using `bellman` and `halo2`. These were deprecated in favor of the current `risc0` and mock-prover approach to better align with the project's focus on sequencing and batching queue dynamics rather than raw cryptographic circuit optimization.

4.6 Submitter

The Submitter acts as the bridge relay, reliably pushing executed batch data from Layer-2 back to the Layer-1 Ethereum network.

- **Language:** Rust.
- **Workflow (Saga Engine):** Subscribes to the Executor stream via a gRPC Client (compiled using `protobuf-compiler`). To ensure reliable interaction with the L1 network despite potential node disconnections or nonce desynchronization,

the Submitter utilizes a Saga Workflow Engine. This state machine, backed by a persistent database, manages reliable retries, idempotency guarantees, and complex error recovery for L1 RPC calls.

- **Data Availability Formatter:** The DA Formatter is responsible for structuring the execution payload according to the configured DA mode before submission. Crucially, it supports encoding state diffs into standard Calldata or generating the necessary KZG commitments for EIP-4844 Blob data representations. Off-chain DA is implemented only as a local simulated stub.
- **Output:** Broadcasts transactions to the L1 Hardhat node via `eth_sendRawTransaction` and records critical latency and gas metrics to `submitter_metrics.json`.

4.7 Smart Contracts

The L1 verification tier is composed of Solidity smart contracts running on a local Hardhat node. A custom Hardhat node image (`npx hardhat node --hostname 0.0.0.0 --port 8545`) is used in the Docker Compose stack to support dev chain behavior and pre-funded accounts.

- **Language:** Solidity.
- **Contract Role:** The `ZKRollupBridge.sol` contract acts as the aggregate root for Layer-1. It delegates DA validation to specific providers (`CalldataDA`, `BlobDA`) and proof verification to a registry of verifiers. It maintains state root commitments and orchestrates deposit and withdrawal flows.
- **Mock Verifier and Real Verifier Scope:** During standard benchmarking sweeps, local L1 interactions bypass cryptographic verification using `MockVerifier.sol` instead of a real verifier, ensuring L1 execution time does not artificially bottleneck the L2 throughput metrics. However, for real proof integration (Stage 5), a real Groth16 Verifier is used. It is critical to note that the mock verifier consumes significantly less gas ($\sim 1,000$ gas) compared to a real Groth16 verifier ($\sim 250,000$ gas), which must be accounted for in the cost models.
- **Testing:** Contracts are strictly required to maintain 100% branch and line coverage via `npx hardhat test`.

4.8 State Synchronization and Nonce Handling

A critical implementation detail of the modular architecture is how state is synchronized across the decoupled components, particularly concerning account nonces.

The Sequencer performs local, pessimistic validation of incoming transactions against a cached state. It ensures that the transaction's nonce matches the account's current nonce in the pool. If a transaction is valid, it is accepted and queued.

However, the Executor acts as the single source of truth for the canonical state-transition logic. When the Executor receives a batch of transactions, its State Transition Function (STF) evaluates them strictly. If a transaction's nonce does not perfectly match the expected canonical state (e.g., due to a skipped nonce), the Executor immediately rejects the transaction.

This strict separation of concerns highlights a systemic vulnerability when global priority sorting policies (such as FeePriority or TimeBoost) are applied. If the Sequencer's scheduling policy reorders transactions purely based on gas price or multi-dimensional scoring without respecting the sender's nonce sequence, it introduces nonce gaps. Consequently, the Executor will reject the out-of-order transactions, resulting in execution failures and empty batches. This architectural disconnect between naive global priority sorting in the Sequencer and strict canonical execution in the Executor explains the scheduler failures observed in the Stage 3 experiments.

4.9 Benchmarking and Data Collection Tools

The benchmarking suite is orchestrated via `run_matrix.sh` and uses Docker Compose profiles like `--profile core` and `--profile bench`. The suite provides an automated environment for configuration-matrix testing.

- **Orchestration:** A suite of bash scripts (e.g., `run_experiment.sh` and `run_pipeline.sh`) manages the lifecycle of the Docker containers, sets configuration variables via `envsubst`, and ensures component synchronization.

4.10 Data Tools

The Data Tools analytics pipeline processes telemetry data by consuming raw JSON/CSV metrics from `benchmark-suite/metrics/` and aggregating/analyzing them using Python scripts (such as `aggregate.py` and `stats.py`) in the `data-tools/` directory. This pipeline requires dependencies such as `pandas` and `matplotlib`.

4.11 Configuration Management

Component configuration is managed via TOML and YAML files. The orchestration scripts map benchmark parameters into these templates using ‘envsubst’. Default values are explicitly exported in the bash environment prior to substitution.

4.12 Logging and Metrics

Rust components use standard tracing and logging crates to provide detailed debug output. Telemetry files (CSV/JSON) are mounted locally via Docker volumes to ensure they persist after container shutdown for the data tools to analyze.

4.13 Integration Challenges

During implementation, several integration challenges were encountered and resolved. The shift to a modular architecture required robust communication protocols, leading to the use of gRPC and Protocol Buffers between the Sequencer, Executor, and Submitter. State synchronization between the Sequencer’s pessimistic cache and the Executor’s canonical state required careful handling to prevent race conditions. Furthermore, managing the volatile local Hardhat RPC required implementing robust retry logic and accurate gas estimation within the Submitter’s Saga engine. Dependency conflicts, particularly between the `nightly-2024-08-01` Rust toolchain and `edition2024` crates in the Executor, required specific workarounds during the build process.

4.14 Development Challenges

One notable development challenge was ensuring the test suite could run cleanly despite upstream dependencies. Specifically, the `risc0_prover` guest execution threw ‘Elf parse errors’ under certain host architectures, which required targeted test bypasses and careful Docker image construction to stabilize the CI pipeline.

CHAPTER 5

EVALUATION AND RESULTS

This chapter details the testing environment, benchmark configurations, controlled variables, and metrics collection methods used to evaluate the modular ZK-Rollup prototype.

5.1 Environment and Orchestration

The experimental environment is strictly local and fully containerized, orchestrated via `docker compose` to ensure consistency and reproducibility across different host machines. The stack is segmented into profiles:

- `--profile core`: Starts the foundational components, including the Hardhat L1 node, the deployer script, Executor, Submitter, and Sequencer.
- `--profile bench`: Triggers the Workload Generator and benchmark harness (`run_experiment.sh`).
- `--profile report`: Triggers the Data Tools analytics pipeline to process raw telemetry into aggregated reports.

5.1.1 Local L1 Configuration

The local L1 network is provided by a custom Docker image (`contracts/Dockerfile.node`) running `npx hardhat node --hostname 0.0.0.0 --port 8545`. This setup is crucial because it provides deterministic, ultra-fast block mining (12-second intervals by default) and pre-funded accounts, allowing the benchmark to saturate the sequencer without being constrained by public testnet block times or faucet limits. Contract deployment is automated via a deployment script (`npx hardhat run scripts/deploy-local.ts`) executed immediately after the node initializes.

When running the local sequencer against a fresh local Hardhat node, the `start_block` in `sequencer.yaml` (or `sequencer.docker.yaml`) must be strictly set to 0 to prevent

the sequencer from attempting to sync historical blocks that do not exist, which would otherwise cause initialization failures.

5.2 Software Stack

The system relies on Docker Engine and Docker Compose. Component-specific toolchains include Python 3.11 for workload generation, Node.js v18 for Hardhat, and Rust (both stable and nightly-2024-08-01 due to executor requirements).

5.2.1 L1 Network Configuration

A custom Docker image (`contracts/Dockerfile.node`) running `npx hardhat node --hostname 0.0.0.0 --port 8545` is utilized as the local L1 network instead of Geth. This provides a deterministic dev-chain environment with pre-funded test accounts, which is necessary for automated testing without the unpredictability and cost of public testnets (like Sepolia). Furthermore, when running against a fresh Hardhat node, the Sequencer is configured with `start_block = 0` to prevent synchronization failures.

5.3 Configurations and Experimental Variables

Component configurations are managed via YAML and TOML templates (e.g., `sequencer.docker.yml`, `sequencer.template.toml`). These templates are processed using `envsubst` to inject environment variables (like `SEQUENCER_BIN`, `L1_RPC_URL`, and `BRIDGE_ADDRESS`). It is crucial that default values are explicitly exported in the bash environment prior to substitution, as the `envsubst` implementation does not support bash fallback syntax.

Table 5.1: Experiment Matrix: Stages 1 through 5

Stage	Focus Area	Key Independent Variables Swept
Stage 1	Fixed Batching & Latency-Cost Trade-off	Max Batch Size (10, 25, 50, 100, 250, 500, 1000)
Stage 2	Adaptive Batching Thresholds	Load Thresholds (Low, Medium, High) under varying offered TPS
Stage 3	Sequencer Scheduling Policies	Scheduling Algorithm (FCFS, FeePriority, TimeBoost, FairBFT, Blob-Packing)
Stage 4	Data Availability (DA) Modes	DA Mode (Calldata, Blob, Offchain), Blob Target Bytes
Stage 5	Prover Backend & Cycle Analysis	Prover Mode (Mock vs RISC0), Target Batch Size (50, 100, 200, 500)

The metrics pipeline scripts (`data-tools/aggregate.py`) require these stages to be executed sequentially, relying on bash scripts inside `benchmark-suite/scripts/` (e.g., `run_experiment.sh`). The pipeline joins metrics by batch ID across the `sequencer_batch_metrics.jsonl`, `executor_batch_metrics.jsonl`, and `submitter_metrics.json` files to build a comprehensive lifecycle view of each transaction batch.

5.4 Benchmark Scenarios

Table 5.2 outlines the specific scenarios executed to answer the research questions.

Table 5.2: Benchmark Scenario Matrix

Scenario	Workload	Batch Policy	DA Mode	Purpose
Cost Amortization	Steady-state	Sweeping (10-500)	Calldata	Observe the trade-off between amortized L1 cost and user queueing latency.
Blob Efficiency	Steady-state	Fixed (100)	Blob (4844)	Compare total transaction cost reduction against the calldata baseline.
Prover Bottleneck	Steady-state	Fixed (100)	Calldata	Sweep parameterized proof delay to observe its impact on end-to-end finalization time.

5.5 Metrics Collection

During each benchmark run, the Workload Generator submits a steady-state Poisson arrival stream of synthetic transactions. Telemetry data is recorded locally within the containers and exported as JSON and CSV files to the `benchmark-suite/metrics/` directory.

Table 5.3: Metric Definitions

Metric	Unit	Definition / Measurement Source
Throughput	TPS	Submitted TPS (from Workload Gen) vs. Committed TPS (measured at the L1 Contract level).
Latency	Seconds	The time from transaction submission to soft finality (batch sealed). Analyzed using Average and P95 metrics.
Finalization Time	Seconds	The time from batch sealing to hard finality (L1 verification receipt).
Cost per Transaction	Gas/L1 ETH	The total L1 gas used for the batch divided by the number of transactions.
DA Footprint	Bytes	The size of the compressed batch payload or blob submitted to L1.

5.6 Reproducibility Setup

The entire benchmark matrix can be executed consistently using the `run_matrix.sh` wrapper. This script automatically tears down the stack between runs, drops the internal SQLite databases to prevent state conflicts, and sequentially steps through the variables defined in `experiments.toml`.

Table 5.4: Reproducibility Commands Table

Command/Script	Purpose
<code>docker compose --profile core up -d</code>	Start the foundationa
<code>npx hardhat run scripts/deploy-local.ts --network localhost</code>	Deploy L1 contracts
<code>benchmark-suite/scripts/run_experiment.sh</code>	Execute a specific be
<code>bash run_pipeline.sh</code> (from repo root)	Run the Python Data

It is important to note that when running the data analytics pipeline, `run_pipeline.sh` must be executed from the repository root directory rather than from inside `benchmark-suite/`, to ensure that relative paths to the `data-tools/` directory resolve correctly. This chapter presents the findings from the automated benchmark runs (Stage 1 through Stage 5). It is important to emphasize that these results are derived from a *controlled benchmarking prototype* using synthetic workloads and parameterized delays. Therefore, the absolute numbers represent a configuration-level, relative comparison of trade-offs rather than a claim of production mainnet performance.

5.6.1 Overview of Evaluation Stages

The empirical results are structured across five distinct stages:

- **Stage 1: Batch Size, Cost, and Latency.** Evaluates the fundamental latency-cost trade-off using fixed batch sizes.
- **Stage 2: Adaptive Batching.** Attempts to evaluate dynamic batch scaling based on mempool load.
- **Stage 3: Sequencer Scheduling Policies.** Evaluates the effect of transaction ordering policies (like TimeBoost and FeePriority) on fairness and throughput.
- **Stage 4: Data Availability.** Analyzes the cost savings of Calldata, Blob, and Offchain DA modes.
- **Stage 5: RISC0 Proving.** Profiles the cycle behavior, step-functions, and fixed overheads of real cryptographic proof generation.

5.6.2 Unsupported and Inconclusive Findings

To maintain rigorous academic integrity, it is crucial to explicitly state which configurations or claims were *not* validated by the experimental results:

- **Adaptive Batching Not Validated:** As detailed in Section 5.10, adaptive batching was not proven effective due to a hysteresis threshold trap.
- **Blob Fill Target Inactive:** Code audits confirmed that the `blob_fill_target` configuration parameter is inactive and unused by the sequencer’s sealing logic.
- **BlobPacking Not Validated:** The BlobPacking heuristic triggered the same global nonce-reordering failures observed in other priority policies, resulting in failed execution.
- **Priority Scheduling Requires Redesign:** True priority scheduling cannot be effectively used without a nonce-aware grouping redesign.
- **Mock Metrics Are Not Production Metrics:** Mock prover and mock verifier results ($\sim 1,000$ gas L1 cost) isolate queueing dynamics but do not reflect the computational bottlenecks or real gas costs ($\sim 250,000$ gas) of a production zkEVM.

5.7 Effect of Batch Size on Latency and Cost (Stage 1)

Stage 1 sweeps evaluated the performance of a Fixed Batching Policy under varying constraints of maximum batch size. Analysis of the raw data reveals the fundamental latency-cost trade-off inherent in rollup design: larger batch sizes reduce per-transaction L1 gas costs by amortizing fixed submission overhead, but exponentially increase mempool queuing delays.

For instance, configuring the sequencer with a batch size of 25 resulted in a low average queue wait time of 1,392 ms, but incurred a relatively high gas cost of approximately 20,549 gas/tx. Conversely, increasing the batch size limit to 500 drastically reduced the amortized gas cost to roughly 19,459 gas/tx, but at the severe penalty of inflating the queue wait time to 74,209 ms.

It is critical to note that the raw Stage 1 metrics exhibited boundary-effect distortions. Specifically, empty tail batches submitted at the end of finite test runs artificially inflated the simple average gas/tx for large configurations. Thus, evaluating the steady-state performance must account for this tail distortion.

5.8 Cost Amortization

Stage 1 and Stage 5 analyses confirmed the theoretical cost amortization curve. Larger batch sizes significantly reduced the amortized Layer-1 gas cost per transaction.

In Stage 1 (using a Mock Verifier with 29,000 fixed gas overhead):

- **Batch Size 25:** True Weighted Gas/Tx was 20,549 gas.
- **Batch Size 100:** True Weighted Gas/Tx was 19,681 gas.
- **Batch Size 500:** True Weighted Gas/Tx was 19,459 gas.

While the gas savings appear small (5%) due to the mock verifier's low overhead, Stage 5 experiments utilizing the real risc0 proof backend demonstrated the massive impact of the real prover fixed cost. The proving wall-clock time was modeled via linear regression as $T(N) = 75s + 5.7s \times N$.

This high fixed setup cost (75 seconds to compile/load the guest ELF and wrap the STARK proof) makes small batches extremely inefficient. At batch size 50, the amortized proving cost was 7.5 seconds per transaction, whereas at batch size 246, it dropped to 6.0 seconds per transaction. A proposed dual-axis plot demonstrating Batch Size vs. Queue Latency & Gas Cost clearly illustrates the diminishing returns of increasing batch size against the penalty of queue wait times.

5.9 Queueing Latency (Soft Finality)

The Stage 1 experiments established a clear relationship between batch size configurations and queueing latency (the time a transaction waits in the sequencer pool before the batch is sealed).

As the timeout interval was increased (from 500 ms to 10,000 ms), the average queue wait time scaled linearly until it hit the ceiling defined by the MAX_BATCH_SIZE. For example:

- **Timeout 500 ms:** Actual queue wait was 256.4 ms.
- **Timeout 2000 ms:** Actual queue wait was 1017.2 ms.
- **Timeout 10000 ms:** The batch filled to the size cap (100 txs) in roughly 5.4 seconds due to the arrival rate, resulting in an actual queue wait of 2937.1 ms instead of the full 10 seconds.

A proposed visualization for these findings is a dual-axis plot showing Timeout on the X-axis, Average Queue Wait Time on the Left Y-Axis, and Average Batch Size on the Right Y-Axis. This plot clearly illustrates how the timeout acts as the primary batch sealing trigger until the size cap is reached.

5.10 Throughput Results and Configuration Anomalies

Initial measurements (Stage 1) captured the baseline throughput of the system. The synthetic Poisson generator successfully offered load to the sequencer up to 100 TPS. However, the committed throughput (measured at the L1 Contract level) was ultimately bounded by the L1 block confirmation time of the local Hardhat node (simulating a 12-second block time). Under heavy load, the sequencer correctly applied backpressure and batched transactions up to the defined MAX_BATCH_SIZE limits (e.g., 100).

5.10.1 Adaptive Batching Failures (Stage 2)

Stage 2 attempted to evaluate an Adaptive Batching policy. However, results showed that adaptive batching was not proven effective. Due to configuration issues during experimental reruns, the system behaved identically to fixed batching. Furthermore, an audit of the threshold logic revealed a "hysteresis threshold trap," where the sequencer's size-threshold trigger would never fire at smaller batch sizes under steady arrival, immediately jumping to larger sizes instead.

5.10.2 Sequencer Scheduling and Nonce Ordering (Stage 3)

Stage 3 evaluated transaction scheduling policies including FCFS, FeePriority, TimeBoost, and BlobPacking. The experiments revealed a systemic flaw: global transaction sorting policies fundamentally break single-account nonce ordering.

Because the sequencer reordered transactions based on bids or blob sizes without respecting account-level nonce constraints, the Executor (which maintains canonical state) rejected out-of-order transactions. This caused permanent nonce gaps, resulting in empty batches (0 executed transactions).

Consequently, these empty batches artificially inflated the gas cost to approximately 112k–115k gas/tx. While TimeBoost showed a higher Jain's fairness index under the tested burst scenario, this result is limited because the scheduler experiments were severely affected by these nonce-ordering failures.

5.11 Data Availability Mode Efficiency (Stage 4)

Stage 4 benchmarks evaluated the performance, L1 gas efficiency, and storage utilization under different Data Availability (DA) Modes: Calldata, Blob (EIP-4844), and Offchain DA.

- **Calldata Footprint:** When using calldata, the Submitter passes the transaction payload directly in the L1 transaction. The cost observed during steady-state traffic was approximately \$0.188 to \$0.194 per transaction, driven by the high cost of storing bytes permanently on Ethereum (16 gas per non-zero byte).
- **Blob DA Efficiency:** Transitioning the DA mode to EIP-4844 blobs significantly minimizes the L1 execution footprint. Because the EVM only verifies a cryptographic commitment to the blob, the L1 footprint is reduced. The cost dropped to approximately \$0.057 per transaction, achieving a massive **~70% cost reduction** compared to Calldata.
- **Offchain DA Efficiency:** Moving to Offchain DA further reduced costs to approximately \$0.033 per transaction, yielding an **~82.5% cost reduction** over Calldata. In this mode, L1 only processes the fixed zk-proof verification and state root update overhead (~115,732 gas per batch). However, Offchain DA requires significantly different trust assumptions, leaning towards a Validium model rather than a pure Rollup.

During these sweeps, testing the `BLOB_TARGET_BYTES` parameter revealed that under FCFS, batch sealing was driven entirely by timeouts, causing larger byte targets to merely dilute the blob utilization ratio. Furthermore, code audits confirmed that the `blob_fill_target` parameter was completely inactive in the sequencer's sealing logic, highlighting a limitation in the prototype's implementation.

5.12 Prover Backend and Real Proof Behavior (Stage 5)

Stage 5 benchmarks replaced the Mock Prover with the real RISC0 Prover Backend (using Groth16 proof compression) to profile cycle behavior and step-function limits independently.

A key discovery is the massive fixed overhead of real proof generation. The compilation and SNARK-wrapping overhead takes approximately **70 to 80 seconds** of wall-clock time, with a marginal proving cost of roughly **5.7 seconds per transaction**.

Therefore, the general proving time model can be approximated as $T(N) = 75 + 5.7N$ seconds.

Because of this high fixed cost, larger batches amortize the overhead more effectively. Proving time improved from roughly 7.5 seconds/tx at batch size 50 down to 6.0 seconds/tx as batch sizes grew to 500.

Furthermore, RISC0 prover execution time does not scale linearly but rather in a step-function governed by segment boundaries (each segment representing 1,048,576 cycles). For example, Batch 49 completed in approximately 333.4 seconds, while crossing a segment boundary caused Batch 50 to spike to 362.3 seconds.

Regardless of the batch size (swept from 3 to 246 executed transactions), the final compressed Groth16 proof size remained perfectly constant at exactly **256 bytes**.

5.12.1 Finalization Time (Hard Finality)

The proof delay heavily dominated the batch finalization time. When the delay was set to 0s, the system was bottlenecked primarily by local database I/O and L1 submission limits. As the delay increased to 30s, the time gap between soft finality (batch sealed) and hard finality (L1 verification) widened dramatically. **Note:** The framework sup-

ports this metric, but the inspected repository snapshot does not include a completed run for this configuration.

5.12.2 Queue Backpressure

High proof delays caused backpressure. While the sequencer could continue accepting transactions and forming batches (soft finality), the pipeline downstream (Executor and Submitter) became compute-bound. This highlights the necessity for highly parallelized prover networks in production rollups to prevent the proof-generation stage from completely throttling L1 settlement throughput.

5.13 Summary of Validated Findings

The observations clearly define the fundamental throughput–latency–cost trade-off inherent in rollup batching policies.

In summary, the empirical results robustly support the following conclusions:

- Larger batches significantly reduce L1 gas/tx but exponentially increase queue latency (Stage 1).

- Blob DA provides an approximate 70% cost reduction over standard Calldata, while Offchain DA provides an 82.5% reduction but alters trust assumptions (Stage 4).
- Real RISC0 proof generation exhibits a massive 70-80 second fixed compilation overhead, highlighting the necessity of large batch amortization (Stage 5).
- RISC0 execution time scales in a step-function based on segment boundaries (Stage 5).
- The final compressed Groth16 proof size remains constant at exactly 256 bytes, ensuring $O(1)$ L1 verification costs regardless of batch size (Stage 5).

Note: The dynamic batching cost model referenced in early literature is analytical and not implemented in the current runtime codebase. The results presented here reflect the empirical behavior of the implemented microservice stack. This chapter synthesizes the experimental results, answering the research questions by discussing the fundamental trade-offs identified during the benchmarking process and reflecting on the practical lessons learned.

5.14 Answering the Research Questions

The following sections address each research question posed at the beginning of the study, mapping the empirical results to the original inquiries regarding batch size, sequencing policies, data availability, and implementation bottlenecks.

5.14.1 RQ1: Impact of Batch Size on Performance and Cost

Batch size fundamentally dictates the throughput–latency–cost trade-off. Larger batches significantly reduce the per-transaction L1 gas cost by amortizing the fixed batch settlement and proof verification overheads over a greater number of transactions. However, this economic efficiency comes at the direct expense of latency, as transactions must wait longer in the sequencer queue before the batch is sealed. Furthermore, when using a real RISC0 prover, larger batches also reduce the amortized proof generation overhead per transaction, mitigating the massive 70-80s fixed compilation delay.

5.14.2 RQ2: Impact of Sequencer Scheduling Policies

The experiments demonstrated that sequencer scheduling policies must be meticulously designed to maintain execution correctness. Naive global sorting policies (like

FeePriority or BlobPacking) completely broke single-account nonce ordering, causing the Executor to reject transactions and producing empty batches. While time-windowed policies like TimeBoost suggest improvements in fairness under burst scenarios, true priority scheduling cannot be implemented without a nonce-aware redesign that groups transactions by account before applying priority heuristics.

5.14.3 RQ3: Cost Implications of Data Availability Mechanisms

Data Availability mode selection is the most significant lever for reducing L1 operational costs. Using EIP-4844 blobs instead of standard calldata resulted in an approximately 70% reduction in per-transaction costs. Transitioning to off-chain DA further reduced costs by approximately 82.5% compared to calldata. However, off-chain DA shifts the security assumptions, meaning cost savings must be weighed against the loss of Ethereum’s native cryptographic data guarantees.

5.14.4 RQ4: Practical Bottlenecks in Implementation

Implementing the modular prototype revealed several practical engineering bottlenecks. The most prominent bottleneck is cryptographic proof generation; real proving introduces massive fixed overhead and step-function cycle limits that are difficult to predict without explicit cycle counting. Furthermore, relying on the local Hardhat RPC for rapid, automated submissions highlighted the volatility of L1 finality and the necessity of robust retry mechanisms (implemented via the Submitter’s Saga pattern). Finally, the disconnect between the Sequencer’s pessimistic cache and the Executor’s strict canonical state highlighted the difficulty of maintaining state synchronization across decoupled microservices.

5.15 The Throughput–Latency–Cost Trade-off

The empirical results observed in Stage 1 and Stage 5 clearly illustrate the fundamental throughput–latency–cost trade-off inherent in Layer-2 rollups. Optimizing for the absolute lowest Layer-1 cost per transaction dictates the use of exceptionally large batches to spread the fixed L1 submission and verification costs. However, this inherently penalizes early-arriving users with high queueing latency as they wait for the batch to fill. Conversely, providing fast soft finality requires smaller, rapidly sealed batches triggered by low timeouts, which drives up the amortized L1 cost per transaction.

5.16 Batching Dynamics and Outliers

Analysis of the Stage 1 queueing distributions revealed a critical insight regarding tail batch latency distortion. When operating with high timeouts, finite test runs created severe tail-end latency outliers. For instance, the final batch in a run with a 200s timeout waited for nearly 200,000 ms because the workload generator stopped offering traffic. This demonstrates that simple arithmetic means are insufficient for measuring sequencer latency; median (P50) and 95th percentile (P95) metrics provide a much more accurate reflection of user experience.

Furthermore, Stage 5 Prover metrics highlighted the step-function nature of zkVM execution segments. Because RISC Zero execution jumps by 140 seconds whenever cycles cross a segment boundary (multiples of 1,048,576 cycles), sequencer batching algorithms should ideally be "cycle-aware." Rather than blindly packing transactions until a hard limit is reached, an optimal sequencer would seal batches just under these segment thresholds to maximize throughput while minimizing proving time penalties.

5.16.1 RQ3: Data Availability Implications

The integration of EIP-4844 blobs fundamentally shifts the economic trade-off curve compared to traditional calldata. As demonstrated by the baseline gas cost of 19,300 marginal gas per transaction using calldata, data costs rapidly become the dominant economic constraint. By drastically reducing the L1 execution data footprint via blobs, rollups can decouple the data submission cost from the execution verification cost. This allows rollups to either operate more profitably at the same batch size or maintain current profitability while significantly reducing batch sizes (thereby dramatically improving user latency and soft finality times).

5.16.2 RQ1: Impact of Batch Size on Cost and Latency

The empirical data explicitly answers the trade-off regarding batch size. While large batch sizes drastically reduce the amortized per-transaction L1 gas cost and the fixed prover setup cost, they directly inflate proof generation latency and transaction queueing times. In scenarios where real cryptographic proof generation dominates the execution time (masking all other systems-level bottlenecks), increasing the batch size distributes the massive fixed setup time across more transactions, but risks causing severe backpressure on the L2 state if the throughput exceeds the prover's linear cycle capacity.

5.17 Design Implications and Trade-offs

The findings from this research carry several critical implications for the design of future ZK-rollup architectures.

- **Use Blob DA when Cost Matters:** The empirical data overwhelmingly supports the adoption of EIP-4844 blobs over calldata to achieve drastic cost reductions. Rollups prioritizing low fees should default to Blob DA.
- **Avoid Small Proving Batches:** Given the massive 70-80s fixed overhead of RISC0 proving, submitting small batches is highly inefficient. Proving should be delayed until a sufficient number of transactions accumulate to amortize this fixed cost.
- **Use Segment-Aware Batching for RISC0:** Because proving latency scales in a step-function based on segment boundaries, sequencers should ideally be "cycle-aware." Sealing a batch just before it crosses a segment boundary optimizes proving time.
- **Preserve Nonce Ordering Before Applying Priority:** The most critical failure observed was the global reordering of transactions. Sequencers must group transactions by sender account and preserve their intra-account nonce order *before* applying inter-account priority sorting (like FeePriority or TimeBoost).
- **Separate Soft Finality from Hard Finality:** Users care about latency, but the system cares about L1 security. Decoupling the Sequencer's soft finality (which can be instantaneous) from the Submitter's hard finality (which must wait for L1 block times and prover overhead) is essential for a responsive user experience.
- **Do Not Use TPS Alone as the Main Metric:** Throughput (TPS) is easily manipulated by submitting empty or artificially small batches. A holistic evaluation must report TPS alongside latency, L1 gas cost, proof generation time, DA bytes, and execution success rate to provide a true picture of system efficiency.

5.17.1 RQ4: Practical Bottlenecks and Trade-offs

Building and benchmarking this prototype revealed severe trade-offs. The high fixed cost of real STARK proving (75s wall-clock setup) forces a choice between acceptable user latency and economic viability.

While this prototype is a controlled research tool, the findings have implications for production rollups:

1. **Dynamic Policies are Necessary:** Static batch size limits are insufficient for volatile real-world traffic. Production sequencers must implement dynamic scheduling policies that balance timeout triggers and size triggers to maintain acceptable user latency during low-traffic periods.
2. **Prover Parallelization:** The backpressure observed during high mock-delay scenarios underscores why production systems invest heavily in massively parallelized, hardware-accelerated prover networks and recursive proof aggregation.
3. **DA is Still Critical:** Even with EIP-4844, the DA footprint remains a core bottleneck. Further compression strategies or reliance on off-chain DA layers (like Celestia) will be necessary for extreme scale.

Table 5.5: Prototype Scope Boundary

Area	Included	Not Included
Execution	Transfer-centric logic	Full EVM/zkEVM
Prover	Benchmark delay	Production proving

Table 5.6: Why Research-Grade Instead of Production-Grade

Dimension	Production ZK-Rollup	Research-Grade Prototype
Goal	Real user deployment	Controlled benchmarking
Execution	Full EVM or custom zkVM	Transfer-centric logic
Prover	Real production pipeline	Mock/parameterized proof-delay
Hardware	GPU/high-RAM/cluster	Student/research hardware
Modifiability	Hard to change safely	Modular, configurable
Metrics	Operational monitoring	TPS, latency, DA bytes, cost
Claim	Production security	Research prototype

CHAPTER 6

CONCLUSIONS AND RECOMMENDATIONS

6.1 Conclusion

This report demonstrated the design, implementation, and evaluation of a modular ZK-Rollup benchmarking prototype, aimed at addressing the critical scalability challenges inherent in Ethereum Layer-1. Unlike production-grade rollups—which prioritize EVM equivalence and reliability at the cost of experimental flexibility—this prototype was explicitly architected to isolate and evaluate distinct performance bottlenecks under controlled, synthetic conditions.

By cleanly separating the sequencing, execution, proving, and submission stages into Rust microservices and providing a Python data-tools pipeline, the framework allowed for a systematic analysis of the fundamental throughput–latency–cost trade-offs in Layer-2 scaling. The empirical observations yielded several key conclusions:

1. **The Batching Dilemma:** Larger batch sizes significantly reduce the amortized Layer-1 gas cost per transaction, but directly penalize early-arriving users by increasing queueing latency (delaying soft finality).
2. **Data Availability Dynamics:** Transitioning from standard transaction call data to EIP-4844 blobs drastically reduces the L1 execution gas footprint. This structural efficiency gain allows rollups to operate more cost-effectively, fundamentally shifting the profitability curve of Layer-2 submission.
3. **Prover Fixed Costs:** Using real RISC Zero STARK proofs demonstrated a high fixed wall-clock setup cost (75 seconds) and execution step-functions that heavily penalize small batches. Furthermore, through parameterized mock proving, the research demonstrated that if proof generation is significantly delayed, the system becomes strictly compute-bound, creating severe downstream backpressure.

Ultimately, the prototype successfully serves its purpose as a reproducible research framework, validating that scalability optimizations cannot be achieved in isolation. Enhancing one dimension (e.g., cost via large batches) inevitably impacts others (e.g., latency or computational overhead).

6.2 Future Work

While this project establishes a robust baseline for evaluating ZK-Rollup configurations, several avenues for future research remain:

- **Fix Adaptive Batching:** Resolve the hysteresis threshold trap identified in Stage 2 to enable a functional dynamic batching heuristic.
- **Implement Nonce-Aware Scheduling:** Redesign the sequencer's transaction pool to group transactions by sender account and preserve strict nonce ordering *before* applying priority sorting, preventing the systemic failures observed in Stage 3.
- **Improve Blob Packing:** Implement functional `blob_fill_target` logic and combine it with the nonce-aware scheduler to maximize EIP-4844 payload efficiency.
- **Use Real Prover Clusters:** Transition from the benchmark-mode mock prover to a fully integrated, hardware-accelerated cryptographic prover cluster to measure absolute proof generation costs under heavy, parallelized load.
- **Trace-Driven Workloads:** Replace the synthetic Poisson-based workload generator with trace-driven inputs based on historical mainnet dataset replays to provide greater external validity.
- **Integrate Real External DA Layers:** Expand the offchain DA scope by fully integrating with external DA networks (such as Celestia or EigenDA) rather than relying on local stubs.
- **Expand State Transition Logic:** Expand the executor from a transfer-centric engine to support richer transaction types and more complex smart contract interactions.
- **Public Testnet Deployment:** Migrate the orchestration environment from local Hardhat instances to public Ethereum testnets to test the framework against real network congestion and block finality variance.

REFERENCE LIST

- [1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system.” <https://bitcoin.org/bitcoin.pdf>, 2008.
- [2] Y. Fu *et al.*, “The blockchain trilemma,” 2024.
- [3] V. Buterin *et al.*, “A next-generation smart contract and decentralized application platform,” *white paper*, vol. 3, no. 37, pp. 2–1, 2014.
- [4] Etherscan, “Ethereum (eth) blockchain explorer.” <https://etherscan.io/>, 2025. Accessed: 2026-05-10.
- [5] Chainspect, “Solana vs ethereum tps comparison.” <https://chainspect.app/compare/solana-vs-ethereum>, 2025. Accessed: 2026-05-10.
- [6] Ethereum Foundation, “Scaling.” <https://ethereum.org/developers/docs/scaling/>, 2025. Accessed: 2026-05-10.
- [7] Ethereum Foundation, “Gas and fees.” <https://ethereum.org/developers/docs/gas/>, 2025. Accessed: 2026-05-10.
- [8] Ethereum Foundation, “Layer 2.” <https://ethereum.org/layer-2/learn/>, 2025. Accessed: 2026-05-10.
- [9] S. Chaliasos, I. Reif, A. Torralba-Agell, J. Ernstberger, A. Kattis, *et al.*, “Analyzing and benchmarking zk-rollups,” in *6th Conference on Advances in Financial Technologies (AFT 2024)*, vol. 316 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 6:1–6:23, 2024.
- [10] E. Ben-Sasson *et al.*, “Scalable, transparent, and post-quantum secure computational integrity,” 2018.
- [11] A. Saif *et al.*, “Data availability in rollups,” 2024.
- [12] Ethereum, “Eip-4844: Shard blob transactions,” 2022.

- [13] L. Lavour *et al.*, “Modular blockchains,” 2023.
- [14] A. I. Sanka and R. C. Cheung, “A systematic review of blockchain scalability: Issues, solutions, analysis and future research,” *Journal of Network and Computer Applications*, vol. 195, p. 103232, 2021.
- [15] A. C. Tran, V. V. Thanh, N. C. Tran, and H. T. Nguyen, “An implementation and evaluation of layer 2 for ethereum with zk-rollup,” in *Lecture Notes in Computer Science*, pp. 57–63, Springer, 2023.
- [16] A. Habib *et al.*, “Bottlenecks in zk-rollups,” 2025.
- [17] Q. Zhou, H. Huang, Z. Zheng, and J. Bian, “Solutions to scalability of blockchain: A survey,” *IEEE Access*, vol. 8, pp. 16440–16455, 2020.
- [18] S. Motepalli *et al.*, “Sequencer policies in rollups,” 2023.
- [19] S. Chaliasos *et al.*, “Benchmarking layer-2 systems,” 2024.
- [20] N. Silva *et al.*, “zksync era public dataset,” 2025.

APPENDIX A

REPOSITORY STRUCTURE

An overview of the essential directories within the RollupX repository:

```
.
|-- benchmark-suite/  # Workload generator and orchestration scripts
|-- contracts/        # L1 Smart Contracts (Solidity/Hardhat)
|-- data-tools/       # Python scripts for telemetry aggregation and plotting
|-- docs/            # System architecture and component documentation
|-- executor/        # Rust executor (state transitions and proving integrat
|-- risc0_prover/     # Experimental RISC0 zkVM prover backend
|-- sequencer/        # Rust sequencer (transaction ingestion and batching)
|-- submitter/        # Rust submitter (DA formatting and L1 interaction)
|-- compose/          # Docker compose configuration files
|-- latex/           # LaTeX sources for reports, papers, and proposals
```

APPENDIX B

BUILD AND RUN GUIDE

This appendix outlines the steps required to build and run the RollupX prototype natively for development purposes (bypassing Docker Compose).

Prerequisites:

- Node.js (v18+) and npm
- Rust (stable and nightly-2024-08-01 toolchains)
- Python 3.11+
- SQLite3 CLI, nasm, and protobuf-compiler

Build Commands:

```
# Install contract dependencies
cd contracts && npm install
npx hardhat compile

# Build core Rust components
cd risc0_prover && cargo +stable build -p rollup_host && cd ..
cd executor && cargo build --ignore-rust-version && cd ..
cd sequencer && cargo build --release && cd ..
cargo build --manifest-path submitter/Cargo.toml
```

Run Commands (in separate terminals):

```
# Terminal A: Start L1 Node
cd contracts && npx hardhat node

# Terminal B: Deploy Contracts
cd contracts && npx hardhat run scripts/deploy-local.ts --network localhost
```

```
# Terminal C: Start Executor
export PROVER_BACKEND=risc0 ... # (Set environment variables)
./executor/target/debug/zksync_state_machine

# Terminal D: Start Submitter
cargo run --manifest-path submitter/Cargo.toml --bin submitter -- \
  --config submitter/config/local-offchain-risc0.yaml

# Terminal E: Start Sequencer
cd sequencer && RUST_LOG=info cargo run --release
```

APPENDIX C

BENCHMARK EXECUTION GUIDE

Running an Automated Benchmark (Dockerized):

```
# Start the core infrastructure
docker compose --profile core up -d

# Execute the specific experiment matrix
bash benchmark-suite/scripts/run_matrix.sh --phase feasibility
```

Collecting and Plotting Metrics:

```
# Trigger the data tools analytics pipeline
bash run_pipeline.sh
```

APPENDIX D

CONFIGURATION REFERENCE

The system behavior is largely controlled by TOML and YAML files. Table D.1 highlights key parameters.

```
# Example sequencer.yaml configuration
server:
  host: "0.0.0.0"
  port: 3000
batching:
  max_batch_size: 100
  timeout_interval_ms: 2000
scheduling:
  policy_type: "FCFS"
  timeboost_window_ms: 5000
```

Figure D.1: Example Configuration Dump for Sequencer

Table D.1: Key Configuration Parameters

Parameter	Location	Default	Effect
max_batch_size	sequencer.toml	100	Triggers a batch seal when the tx pool reaches this size.
timeout_interval_ms	sequencer.toml	5000	Triggers a batch seal if the interval expires, regardless of size.
policy_type	sequencer.toml	FCFS	Determines the transaction ordering algorithm (e.g., FCFS or FeePriority).
da_mode	submitter.yaml	calldata	Swaps the L1 data posting mechanism between standard calldata and EIP-4844 blobs.

APPENDIX E

METRICS OUTPUT SCHEMA

The Workload Generator outputs a JSON file (`workload_<exp_id>.json`) capturing the global experiment parameters and a CSV file (`tx_log_<run_id>.csv`) tracking individual transactions.

tx_log schema:

- `tx_hash`: Unique transaction identifier.
- `submit_time`: Timestamp (ms) when the workload generator sent the request.
- `status`: Success or Failure.
- `latency_ms`: The Round-Trip Time observed by the client (approximating soft finality).

```
{  
  "batch_id": 1,  
  "l1_tx_hash": "0xabc...",  
  "gas_used": 115732,  
  "da_mode": "blob",  
  "total_cost_eth": 0.0001  
}
```

Figure E.1: Example JSON Dump for Submitter Metrics

The Submitter outputs `submitter_metrics.json`, containing:

- `batch_id`: Internal ID.
- `l1_tx_hash`: The Ethereum transaction hash.
- `gas_used`: The exact L1 gas consumed by the verification transaction.
- `da_mode`: The DA formatting used (Calldata/Blob).
- `total_cost_eth`: The calculated economic cost of the submission.

APPENDIX F

IMPORTANT ALGORITHMS

This appendix provides simplified pseudocode for the Sequencer's core batch orchestration logic, demonstrating the interaction between the size trigger and the timeout trigger.

Algorithm: Batch Orchestrator Loop

Input: max_batch_size, timeout_interval

State: current_batch_txs = [], last_seal_time = now()

loop indefinitely:

 // Pull transactions from the pool ordered by the active policy

 tx = tx_pool.pop()

 if tx is not null:

 current_batch_txs.append(tx)

 time_elapsed = now() - last_seal_time

 if current_batch_txs.length >= max_batch_size OR time_elapsed >= timeout_in

 if current_batch_txs.length > 0:

 seal_and_dispatch(current_batch_txs)

 current_batch_txs = []

 last_seal_time = now()

 sleep(tick_duration)